



Threagile

Agile Threat Modeling

Threat Model Report

VaultNote Threat Model

18 April 2026

VaultNote Security Team

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 12 Initial Risks in 6 Categories	5
Risk Mitigation	6
Asset Register	7
Impact Analysis of 12 Remaining Risks in 6 Categories	12
Application Overview	13
Data-Flow Diagram	14
Security Requirements	16
Abuse Cases	17
Tag Listing	18
STRIDE Classification of Identified Risks	23
Assignment by Function	25
RAA Analysis	27
Data Mapping	30
Out-of-Scope Assets: 0 Assets	31
Potential Model Failures: 0 / 0 Risks	32
Questions: 0 / 0 Questions	33

Risks by Vulnerability category

Identified Risks by Vulnerability category	34
Stage III - Undocumented Application Entry Point: 5 / 5 Risks	35
Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks	37
Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk	40
Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point: 1 / 1 Risk	42
Stage VII - Internet-Reachable Datastore With No Mitigation: 2 / 2 Risks	45
Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1 / 1 Risk	47

Risks by Technical Asset

Identified Risks by Technical Asset	49
API Server: 1 / 1 Risk	50
AWS API Gateway: 1 / 1 Risk	54
AWS ECR Container Registry: 2 / 2 Risks	56
AWS SES: 1 / 1 Risk	59
Browser SPA: 1 / 1 Risk	61
GitHub Actions Build Pipeline: 1 / 1 Risk	63
LLM Note Summarizer: 2 / 2 Risks	66
Nginx Reverse Proxy: 1 / 1 Risk	69

VaultNote Source Repository: 1 / 1 Risk	72
VaultNote Vector Store: 1 / 1 Risk	74
AWS RDS PostgreSQL: 0 / 0 Risks	77
AWS S3 Notes Bucket: 0 / 0 Risks	79
ECS Container Platform: 0 / 0 Risks	81
Lambda Email Notifier: 0 / 0 Risks	83
MinIO Object Storage: 0 / 0 Risks	85
Note RAG Pipeline: 0 / 0 Risks	87
PostgreSQL Database: 0 / 0 Risks	90
Redis Cache: 0 / 0 Risks	92

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	94
AI Model Responses: 2 / 2 Risks	95
Build Artifacts: 4 / 4 Risks	96
Embedding Vectors: 3 / 3 Risks	97
Note Content: 7 / 7 Risks	98
File Attachments: 4 / 4 Risks	99
Notification Payloads: 1 / 1 Risk	100
Session Tokens: 1 / 1 Risk	101
User Credentials: 4 / 4 Risks	102

Trust Boundaries

AI Services	103
Application Network	103
AWS Cloud	103
Data Tier	104
DMZ	104
External CI/CD	104
Internet	104

Shared Runtime

Docker Host	106
-------------	-----

About Threagile

Risk Rules Checked by Threagile	107
Disclaimer	108

Management Summary

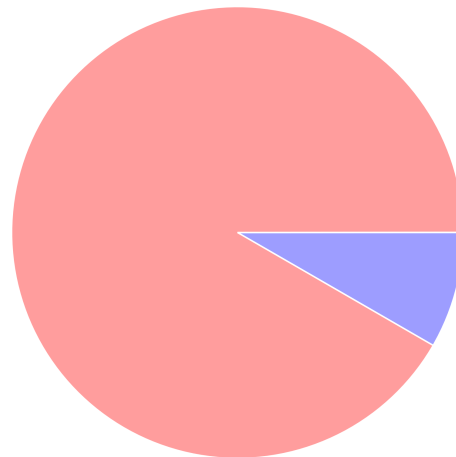
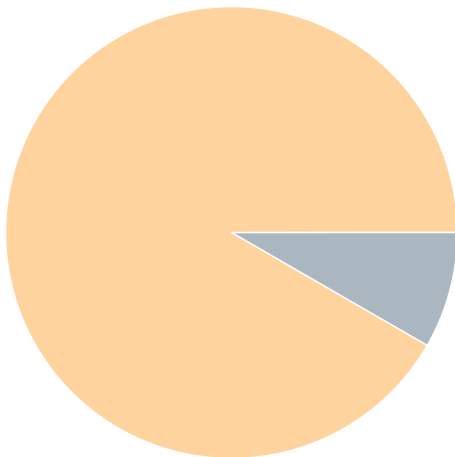
Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **12 initial risks** in **6 categories** have been identified during the threat modeling process:

0 critical risk
0 high risk
11 elevated risk
0 medium risk
1 low risk

11 unchecked
0 in discussion
0 accepted
1 in progress
0 mitigated
0 false positive



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 12 Initial Risks in 6 Categories

The most prevalent impacts of the **12 initial risks** (distributed over **6 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Stage III - Undocumented Application Entry Point: 5 Initial Risks - Exploitation** likelihood is *Likely with Medium* impact.

Undocumented entry points are excluded from threat analysis, leaving unknown attack surface that attackers can exploit without detection or mitigation.

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 Initial Risks - Exploitation** likelihood is *Likely with Medium* impact.

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Elevated: **Stage V - No Rate Limiting on Internet-Exposed API: 1 Initial Risk - Exploitation** likelihood is *Likely with High* impact.

Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Elevated: **Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point: 1 Initial Risk - Exploitation** likelihood is *Likely with High* impact.

Full account takeover, lateral movement into internal systems, and data exfiltration. Credential stuffing attacks against password-only endpoints succeed at statistically predictable rates given leaked credential databases.

Elevated: **Stage VII - Internet-Reachable Datastore With No Mitigation: 2 Initial Risks - Exploitation** likelihood is *Likely with Very High* impact.

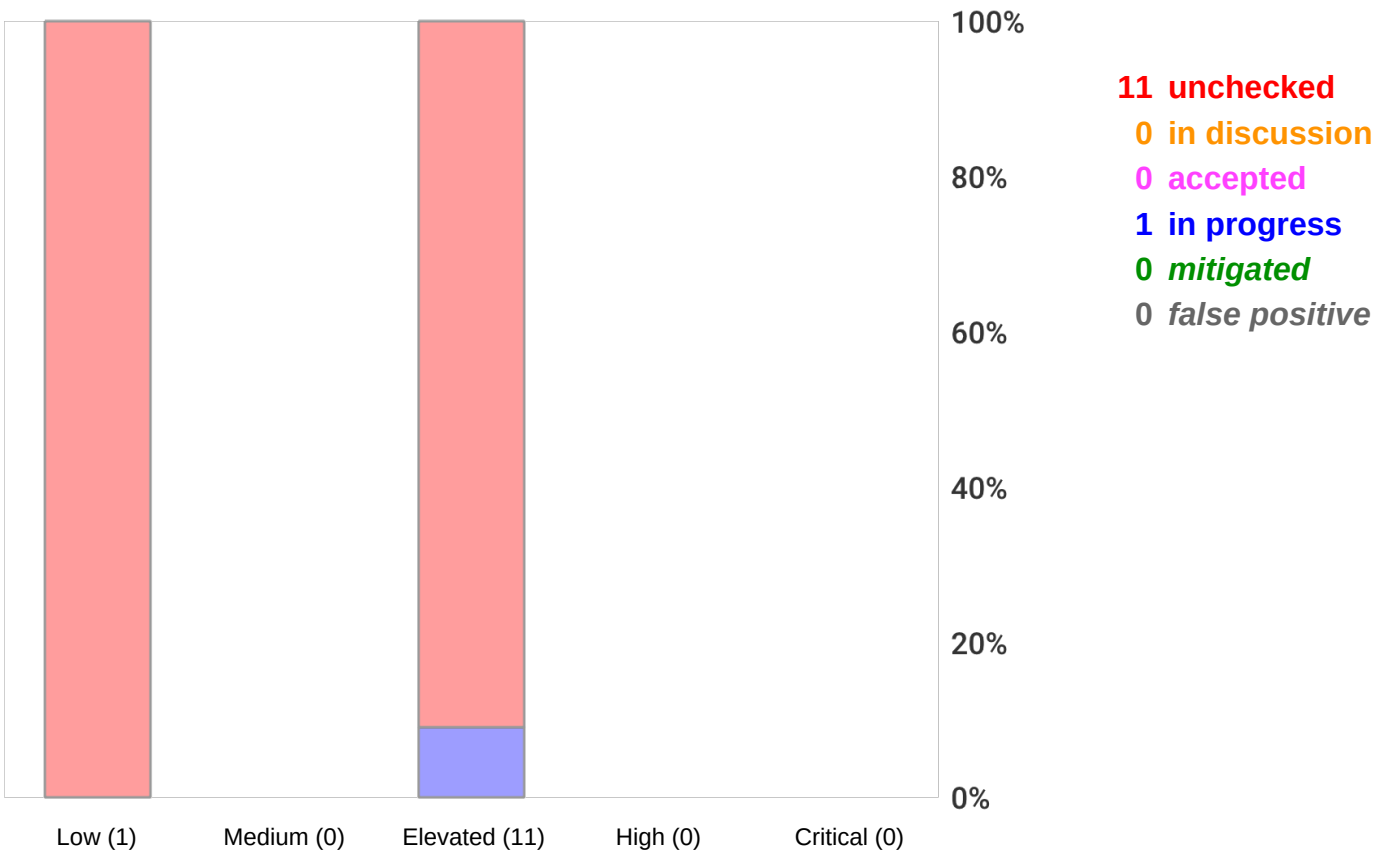
Mass data exfiltration of all stored records, ransomware encryption of datastores, complete loss of data integrity, and regulatory breach notification obligations.

Low: **Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1 Initial Risk - Exploitation** likelihood is *Unlikely with Low* impact.

Without explicit threat actor profiles and attack scenarios, mitigations are chosen reactively rather than proactively, leaving systematic threat actor capabilities unaddressed.

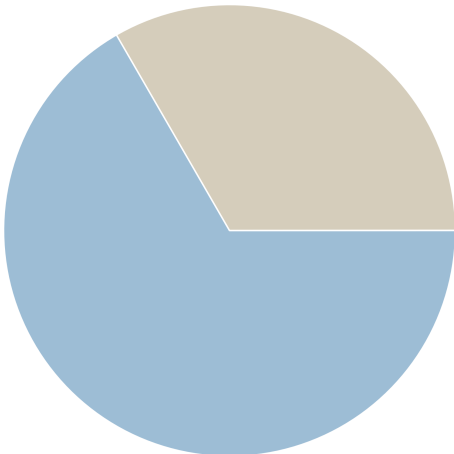
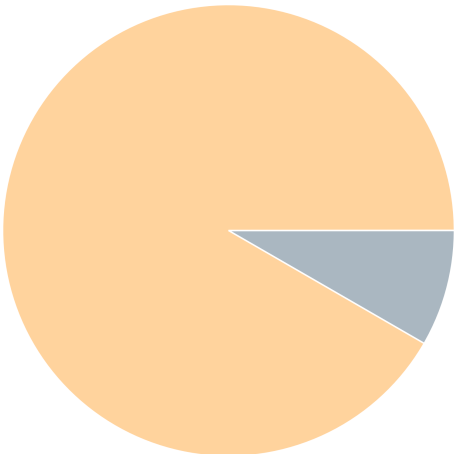
Risk Mitigation

The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following 12 remain unmitigated:

- 0 unmitigated critical risk
- 0 unmitigated high risk
- 11 unmitigated elevated risk
- 0 unmitigated medium risk
- 1 unmitigated low risk
- 0 business side related
- 8 architecture related
- 0 development related
- 4 operations related



Asset Register

Technical Assets

API Server

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

AWS ECR Container Registry

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

AWS RDS PostgreSQL

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

AWS S3 Notes Bucket

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

AWS SES

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Browser SPA

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

ECS Container Platform

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without

signature verification.

GitHub Actions Build Pipeline

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded `AWS_SECRET_ACCESS_KEY` in workflow environment variables.

LLM Note Summarizer

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Lambda Email Notifier

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

MinIO Object Storage

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Note RAG Pipeline

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

PostgreSQL Database

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Redis Cache

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

VaultNote Source Repository

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

VaultNote Vector Store

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Data Assets

AI Model Responses

LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

Build Artifacts

Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

Embedding Vectors

High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

File Attachments

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

Note Content

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

Notification Payloads

Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

Session Tokens

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

User Credentials

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

Impact Analysis of 12 Remaining Risks in 6 Categories

The most prevalent impacts of the **12 remaining risks** (distributed over **6 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Stage III - Undocumented Application Entry Point: 5 Remaining Risks - Exploitation** likelihood is *Likely with Medium* impact.

Undocumented entry points are excluded from threat analysis, leaving unknown attack surface that attackers can exploit without detection or mitigation.

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 Remaining Risks - Exploitation** likelihood is *Likely with Medium* impact.

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Elevated: **Stage V - No Rate Limiting on Internet-Exposed API: 1 Remaining Risk - Exploitation** likelihood is *Likely with High* impact.

Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Elevated: **Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point: 1 Remaining Risk - Exploitation** likelihood is *Likely with High* impact.

Full account takeover, lateral movement into internal systems, and data exfiltration. Credential stuffing attacks against password-only endpoints succeed at statistically predictable rates given leaked credential databases.

Elevated: **Stage VII - Internet-Reachable Datastore With No Mitigation: 2 Remaining Risks - Exploitation** likelihood is *Likely with Very High* impact.

Mass data exfiltration of all stored records, ransomware encryption of datastores, complete loss of data integrity, and regulatory breach notification obligations.

Low: **Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1 Remaining Risk - Exploitation** likelihood is *Unlikely with Low* impact.

Without explicit threat actor profiles and attack scenarios, mitigations are chosen reactively rather than proactively, leaving systematic threat actor capabilities unaddressed.

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

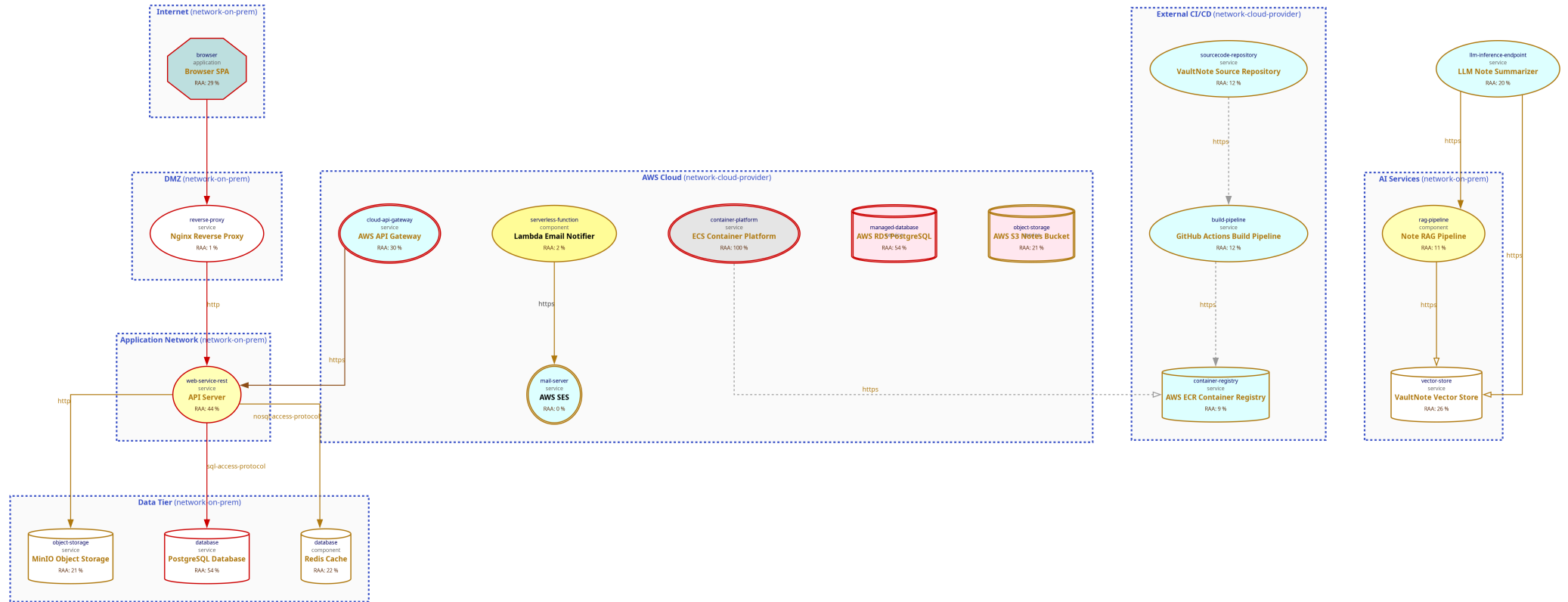
Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.

Data-Flow Diagram - VaultNote Threat Model



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

ai

LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store, AI Model Responses, Embedding Vectors, AI Services

api-gateway

AWS API Gateway

aws

AWS API Gateway, AWS ECR Container Registry, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS Cloud

cache

Redis Cache

ci

GitHub Actions Build Pipeline

cloud-native

AWS API Gateway, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, LLM Note Summarizer, Lambda Email Notifier, AWS Cloud

container-runtime

Docker Host

criticality-high

AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

docker

Docker Host

ecr

AWS ECR Container Registry

ecs

ECS Container Platform

email

AWS SES

embeddings

VaultNote Vector Store, Embedding Vectors

express

API Server

fargate

ECS Container Platform

git

VaultNote Source Repository

github

VaultNote Source Repository

github-actions

GitHub Actions Build Pipeline

has-automated-backup

AWS RDS PostgreSQL

information-asset-container

AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

intentional-misconfiguration

MinIO File Storage, MinIO Object Storage, HTTP to API Server

jwt

API Server

lambda

Lambda Email Notifier

langchain

Note RAG Pipeline

llm

LLM Note Summarizer

minio

MinIO Object Storage

nginx

Nginx Reverse Proxy

nodejs

API Server

notifications

Lambda Email Notifier

object-storage

MinIO Object Storage

openai

LLM Note Summarizer

pii

Notification Payloads

postgresql

AWS RDS PostgreSQL, PostgreSQL Database

qdrant

VaultNote Vector Store

rag

Note RAG Pipeline

rds

AWS RDS PostgreSQL

react

Browser SPA

redis

Redis Cache

relational

PostgreSQL Database

s3

AWS S3 Notes Bucket, MinIO Object Storage

ses

AWS SES

session-store

Redis Cache

spa

Browser SPA

supply-chain

AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository, Build Artifacts, External CI/CD

third-party-shared

AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database

tls-termination

Nginx Reverse Proxy

trike-actor

Browser SPA

trike-actor-untrusted

Browser SPA

trike-trust-level-admin

API Server

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **12 potential risks** have been identified during the threat modeling process of which **1 in the Spoofing** category, **0 in the Tampering** category, **0 in the Repudiation** category, **2 in the Information Disclosure** category, **3 in the Denial of Service** category, and **6 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

Elevated: **Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point: 1 / 1 Risk** - Exploitation likelihood is *Likely with High* impact.

PASTA Stage VI models how attackers simulate attacks against identified entry points.

Internet-exposed entry points with weak authentication (none or password-only) are prime targets for credential stuffing, phishing, and account takeover attacks. High-value or sensitive entry points should require MFA or hardware tokens.

Tampering

n/a

Repudiation

n/a

Information Disclosure

Elevated: **Stage VII - Internet-Reachable Datastore With No Mitigation: 2 / 2 Risks** - Exploitation likelihood is *Likely with Very High* impact.

PASTA Stage VII evaluates residual risk. A datastore (database, cache, object store) that can be reached from the internet, either directly or via a chain of HTTP/plain protocol hops, with no network segmentation represents one of the highest-risk configurations in any architecture. Attackers who compromise any internet-facing component gain a direct path to sensitive stored data.

Denial of Service

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks** - Exploitation likelihood is *Likely with Medium* impact.

PASTA Stage V identifies vulnerabilities in the attack surface. Custom-developed services that process authentication credentials or session tokens and have outgoing communication links without rate limiting are vulnerable to credential stuffing, brute-force, bulk data exfiltration, and resource exhaustion. Global API rate limits are too coarse to prevent targeted abuse of individual

high-value operations such as login, password reset, or bulk data exports.

Elevated: Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk - Exploitation
likelihood is *Likely* with *High* impact.

PASTA Stage V identifies weaknesses that enable exploitation. An internet-accessible API endpoint without rate limiting is vulnerable to brute-force credential attacks, credential stuffing, API abuse, and denial-of-service amplification. Rate limiting is a fundamental control for any internet-exposed API.

Elevation of Privilege

Elevated: Stage III - Undocumented Application Entry Point: 5 / 5 Risks - Exploitation
likelihood is *Likely* with *Medium* impact.

PASTA Stage III requires decomposing the application into entry points so that threats can be mapped against known attack surfaces. A technical asset reachable from the internet or marked as internet-exposed that has no `entry_point_type` declared represents an undocumented entry point that cannot be systematically analysed for threats.

Low: Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1 / 1 Risk - Exploitation
likelihood is *Unlikely* with *Low* impact.

PASTA Stage I defines business and security objectives and maps them to threat actors. An architecture with internet-exposed entry points but no documented threat scenarios means that adversarial thinking has not been applied: the team has not specified who might attack, how, and with what capabilities.

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **12 potential risks** have been identified during the threat modeling process of which **0 should be checked by Business Side**, **8 should be checked by Architecture**, **0 should be checked by Development**, and **4 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

n/a

Architecture

Elevated: **Stage III - Undocumented Application Entry Point: 5 / 5 Risks - Exploitation** likelihood is *Likely with Medium* impact.

Declare entry_point_type on every internet-exposed technical asset (api, web_ui, cli, file_upload, webhook, cron, or internal_rpc). Maintain an entry point register and review it as part of each release cycle.

Elevated: **Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point: 1 / 1 Risk - Exploitation** likelihood is *Likely with High* impact.

Require MFA for all user-facing internet entry points handling sensitive data. Declare requires_authentication_strength as mfa or hardware on the technical asset. Consider phishing-resistant authentication (WebAuthn, FIDO2) for high-value assets.

Elevated: **Stage VII - Internet-Reachable Datastore With No Mitigation: 2 / 2 Risks - Exploitation** likelihood is *Likely with Very High* impact.

Place all datastores in a dedicated backend trust boundary with no direct internet access. Ensure all communication links between internet-facing assets and datastores go through the application tier (no direct datastore exposure). Apply defense-in-depth with connection-level authentication and network ACLs.

Development

n/a

Operations

Elevated: **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting: 2 / 2 Risks - Exploitation** likelihood is *Likely with Medium* impact.

Apply rate limiting at the operation level using token-bucket or sliding-window algorithms, enforced per-user for authenticated requests and per-IP for unauthenticated requests. Set rate_limited: true on every communication link once per-operation limits are implemented and

verified.

Elevated: Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk - Exploitation
likelihood is *Likely* with *High* impact.

Add rate limiting at the API gateway or application layer (token bucket, sliding window). Apply stricter limits on authentication endpoints. Set `rate_limited` to true on the corresponding communication links after implementation.

Low: Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1 / 1 Risk -
Exploitation likelihood is *Unlikely* with *Low* impact.

Add a `threat_scenarios` section to the model with at least one scenario per distinct threat actor type (e.g., script-kiddie, insider, nation-state). Each scenario should document `entry_assets`, `kill_chain_steps`, and `mitigated_by` controls.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

ECS Container Platform: RAA 100%

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

AWS RDS PostgreSQL: RAA 54%

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (`storage_encrypted = false` in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

PostgreSQL Database: RAA 54%

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 44%

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway: RAA 30%

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Browser SPA: RAA 29%

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

VaultNote Vector Store: RAA 26%

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Redis Cache: RAA 22%

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

AWS S3 Notes Bucket: RAA 21%

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

MinIO Object Storage: RAA 21%

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

LLM Note Summarizer: RAA 20%

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

GitHub Actions Build Pipeline: RAA 12%

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

VaultNote Source Repository: RAA 12%

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Note RAG Pipeline: RAA 11%

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the

vector store are injected into context without relevance threshold or content sanitisation.

AWS ECR Container Registry: RAA 9%

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

Lambda Email Notifier: RAA 2%

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Nginx Reverse Proxy: RAA 1%

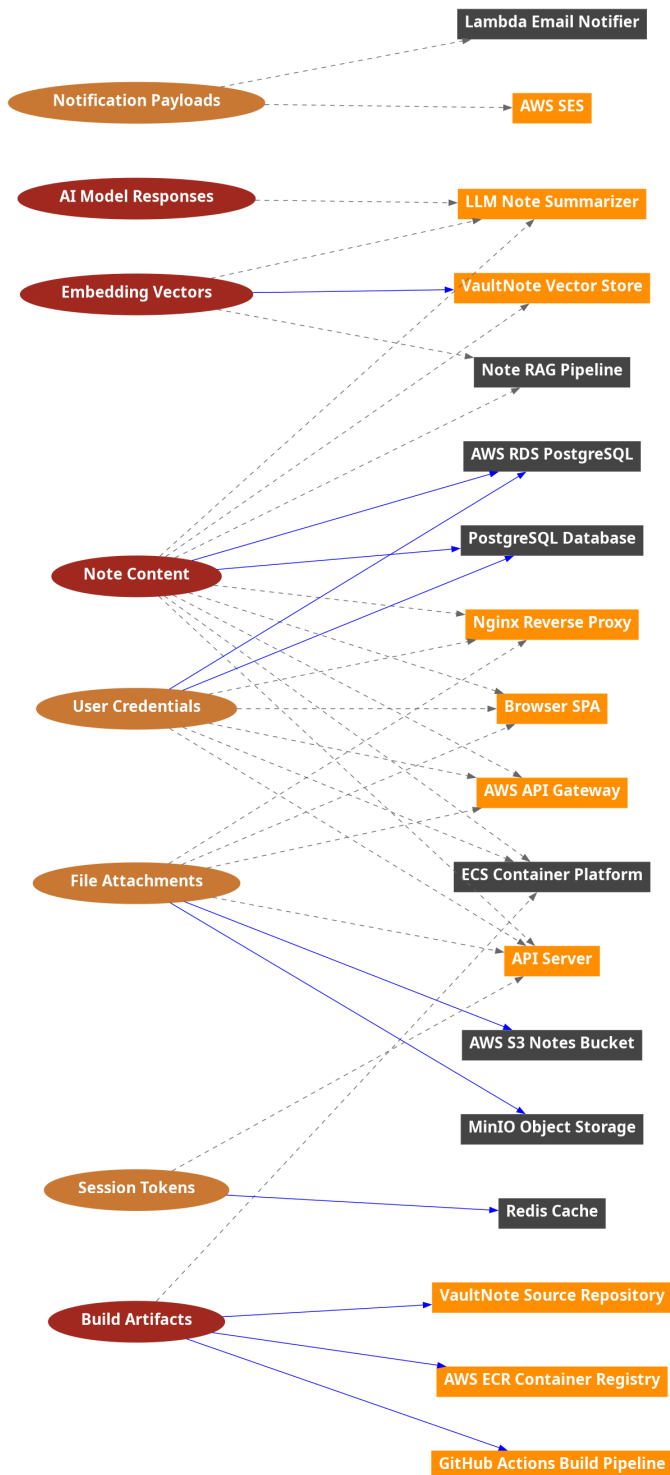
Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

AWS SES: RAA 0%

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 0 / 0 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

No potential model failures have been identified.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **12 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 1 as low.**

These risks are distributed across **6 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Stage III - Undocumented Application Entry Point: 5 / 5 Risks

Description (Elevation of Privilege): [CWE 693](#)

PASTA Stage III requires decomposing the application into entry points so that threats can be mapped against known attack surfaces. A technical asset reachable from the internet or marked as internet-exposed that has no `entry_point_type` declared represents an undocumented entry point that cannot be systematically analysed for threats.

Impact

Undocumented entry points are excluded from threat analysis, leaving unknown attack surface that attackers can exploit without detection or mitigation.

Detection Logic

In-scope technical assets marked `internet=true` or reachable from an internet trust boundary that do not have an `entry_point_type` field declared.

Risk Rating

Medium baseline; elevated when the asset processes sensitive data or requires no authentication.

False Positives

Internal infrastructure assets (databases, caches) that happen to be reachable across a network perimeter but are not themselves application entry points.

Mitigation (Architecture): Document all application entry points

Declare `entry_point_type` on every internet-exposed technical asset (`api`, `web_ui`, `cli`, `file_upload`, `webhook`, `cron`, or `internal_rpc`). Maintain an entry point register and review it as part of each release cycle.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Do all internet-exposed technical assets have an `entry_point_type` declared?

Risk Findings

The risk **Stage III - Undocumented Application Entry Point** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Undocumented Entry Point: AWS API Gateway is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@aws-api-gateway](#)

Unchecked

Undocumented Entry Point: AWS ECR Container Registry is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@container-registry](#)

Unchecked

Undocumented Entry Point: AWS SES is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@aws-ses](#)

Unchecked

Undocumented Entry Point: GitHub Actions Build Pipeline is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@build-pipeline](#)

Unchecked

Undocumented Entry Point: VaultNote Source Repository is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@source-repo](#)

Unchecked

Stage V - Custom-Developed Service Without Per-Operation Rate Limiting:

Description (Denial of Service): [CWE 770](#)

PASTA Stage V identifies vulnerabilities in the attack surface. Custom-developed services that process authentication credentials or session tokens and have outgoing communication links without rate limiting are vulnerable to credential stuffing, brute-force, bulk data exfiltration, and resource exhaustion. Global API rate limits are too coarse to prevent targeted abuse of individual high-value operations such as login, password reset, or bulk data exports.

Impact

Attackers can enumerate users, brute-force credentials, bulk-download sensitive data, or exhaust backend resources by sending unlimited requests to unthrottled operations. Business process integrity degrades when operations can be repeated without throttling.

Detection Logic

In-scope custom-developed process-type technical assets that process authentication data (user-credentials or session-tokens) and have at least one outgoing communication link where `rate_limited` is false and usage is not devops.

Risk Rating

Medium baseline. Elevated for assets with internet-exposed entry points or those processing strictly-confidential authentication credentials.

False Positives

Internal service-to-service links that are not reachable from untrusted callers, or where rate limiting is enforced by an upstream proxy (e.g., Nginx `limit_req`). Set `rate_limited: true` on the link or change usage to devops to suppress.

Mitigation (Operations): Add per-operation rate limiting to all communication links in the authentication and data flow paths

Apply rate limiting at the operation level using token-bucket or sliding-window algorithms, enforced per-user for authenticated requests and per-IP for unauthenticated requests. Set `rate_limited: true` on every communication link once per-operation limits are implemented and verified.

ASVS Chapter: [V4 - Access Control Verification Requirements](#)

Cheat Sheet: [Denial_of_Service_Cheat_Sheet](#)

Check

Is `rate_limited` set to true on every communication link that participates in authentication or sensitive data processing? Are per-user limits enforced in addition to any global API limits?

Risk Findings

The risk **Stage V - Custom-Developed Service Without Per-Operation Rate Limiting** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **API Server** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@api-server](#)

Unchecked

Missing Per-Operation Rate Limiting: custom service **Browser SPA** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@browser-spa](#)

Unchecked

Stage V - No Rate Limiting on Internet-Exposed API: 1 / 1 Risk

Description (Denial of Service): [CWE 770](#)

PASTA Stage V identifies weaknesses that enable exploitation. An internet-accessible API endpoint without rate limiting is vulnerable to brute-force credential attacks, credential stuffing, API abuse, and denial-of-service amplification. Rate limiting is a fundamental control for any internet-exposed API.

Impact

Brute-force attacks against authentication endpoints, mass data harvesting via enumeration, DDoS amplification, and billing abuse on pay-per-use endpoints.

Detection Logic

In-scope technical assets with entry_point_type of api or web_ui that receive communication links from internet-exposed assets where rate_limited is false.

Risk Rating

Elevated for authentication endpoints; medium for general data APIs. High when the API has no authentication at all.

False Positives

Internal APIs behind a WAF or API gateway that provides rate limiting at the infrastructure layer; document and mark rate_limited as true on the comm links.

Mitigation (Operations): Implement rate limiting on all internet-exposed API entry points

Add rate limiting at the API gateway or application layer (token bucket, sliding window). Apply stricter limits on authentication endpoints. Set rate_limited to true on the corresponding communication links after implementation.

ASVS Chapter: [V4 - Access Control Verification Requirements](#)

Cheat Sheet: [Denial of Service Cheat Sheet](#)

Check

Are all internet-exposed API communication links rate-limited?

Risk Findings

The risk **Stage V - No Rate Limiting on Internet-Exposed API** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Rate Limiting: internet-exposed API **Nginx Reverse Proxy** has communication links without rate limiting: Exploitation likelihood is *Likely* with *High* impact.

[no-rate-limiting-public-api@nginx-proxy](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-120

Rate limiting via nginx limit_req_zone is planned for sprint 21. Current mitigation: account lockout after 10 failed login attempts implemented in the API layer. Full IP-level rate limiting is a VAULT-120 blocker before production launch.

Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point:

Description (Spoofing): [CWE 287](#)

PASTA Stage VI models how attackers simulate attacks against identified entry points. Internet-exposed entry points with weak authentication (none or password-only) are prime targets for credential stuffing, phishing, and account takeover attacks. High-value or sensitive entry points should require MFA or hardware tokens.

Impact

Full account takeover, lateral movement into internal systems, and data exfiltration. Credential stuffing attacks against password-only endpoints succeed at statistically predictable rates given leaked credential databases.

Detection Logic

Internet-exposed technical assets classified as entry points where `requires_authentication_strength` is not `mfa` or `hardware` and the asset handles confidential or strictly-confidential data.

Risk Rating

High for entry points storing or processing strictly-confidential data with no MFA. Elevated for confidential data with password-only authentication.

False Positives

Public APIs serving non-sensitive, non-authenticated data (e.g., public search) do not require strong authentication; ensure confidentiality is set to public.

Mitigation (Architecture): Strengthen authentication on all internet-exposed entry points

Require MFA for all user-facing internet entry points handling sensitive data. Declare `requires_authentication_strength` as `mfa` or `hardware` on the technical asset. Consider phishing-resistant authentication (WebAuthn, FIDO2) for high-value assets.

ASVS Chapter: [V2 - Authentication Verification Requirements](#)

Cheat Sheet: [Authentication Cheat Sheet](#)

Check

Do all internet-exposed entry points with confidential or higher data have

requires_authentication_strength set to mfa or hardware?

Risk Findings

The risk **Stage VI - Weak Authentication Strength on Internet-Exposed Entry Point** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Weak Authentication: internet-exposed entry point **LLM Note Summarizer** lacks strong authentication for confidential data: Exploitation likelihood is *Likely* with *High* impact.

[weak-auth-on-entry-point@llm-summarizer](#)

Unchecked

Stage VII - Internet-Reachable Datastore With No Mitigation: 2 / 2 Risks

Description (Information Disclosure): [CWE 284](#)

PASTA Stage VII evaluates residual risk. A datastore (database, cache, object store) that can be reached from the internet, either directly or via a chain of HTTP/plain protocol hops, with no network segmentation represents one of the highest-risk configurations in any architecture. Attackers who compromise any internet-facing component gain a direct path to sensitive stored data.

Impact

Mass data exfiltration of all stored records, ransomware encryption of datastores, complete loss of data integrity, and regulatory breach notification obligations.

Detection Logic

In-scope datastore technical assets that have any incoming communication link originating from an internet-exposed technical asset.

Risk Rating

Critical when the datastore holds strictly-confidential data; high otherwise.

False Positives

Edge datastores intentionally internet-accessible (e.g., CDN origin) with documented justification, no sensitive data, and comprehensive access controls.

Mitigation (Architecture): Segment datastores from internet-facing components

Place all datastores in a dedicated backend trust boundary with no direct internet access. Ensure all communication links between internet-facing assets and datastores go through the application tier (no direct datastore exposure). Apply defense-in-depth with connection-level authentication and network ACLs.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Database Security Cheat Sheet](#)

Check

Are all datastores behind at least one application-tier trust boundary?

Risk Findings

The risk **Stage VII - Internet-Reachable Datastore With No Mitigation** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Internet-Reachable Datastore: AWS ECR Container Registry can be reached from an internet-exposed asset with no application-tier separation: Exploitation likelihood is *Likely* with *Very High* impact.

[internet-reachable-datastore@container-registry](#)

Unchecked

Internet-Reachable Datastore: VaultNote Vector Store can be reached from an internet-exposed asset with no application-tier separation: Exploitation likelihood is *Likely* with *Very High* impact.

[internet-reachable-datastore@vector-store](#)

Unchecked

Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points: 1

Description (Elevation of Privilege): [CWE 693](#)

PASTA Stage I defines business and security objectives and maps them to threat actors. An architecture with internet-exposed entry points but no documented threat scenarios means that adversarial thinking has not been applied: the team has not specified who might attack, how, and with what capabilities.

Impact

Without explicit threat actor profiles and attack scenarios, mitigations are chosen reactively rather than proactively, leaving systematic threat actor capabilities unaddressed.

Detection Logic

Internet-exposed technical assets with a declared entry_point_type but no matching threat scenario that lists them as an entry asset.

Risk Rating

Low severity from a technical standpoint but indicates a process gap that increases risk of systematic coverage omissions.

False Positives

Models in early development where threat scenario documentation is tracked externally in a threat register; provide a reference in the management summary.

Mitigation (Operations): Define threat scenarios for all internet-exposed entry points

Add a threat_scenarios section to the model with at least one scenario per distinct threat actor type (e.g., script-kiddie, insider, nation-state). Each scenario should document entry_assets, kill_chain_steps, and mitigated_by controls.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Threat Modeling Cheat Sheet](#)

Check

Does the model have at least one threat_scenarios entry for each internet-facing entry point?

Risk Findings

The risk **Stage I - No Threat Scenarios Defined for Internet-Exposed Entry Points** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Low Risk Severity

Missing Threat Scenario: internet entry point **LLM Note Summarizer** has no documented threat scenario: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-threat-scenario@llm-summarizer](#)

Unchecked

Identified Risks by Technical Asset

In total **12 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 1 as low.**

These risks are distributed across **18 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 1 / 1 Risk

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **API Server** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@api-server](#)

Unchecked

Asset Information

ID:	api-server
Type:	process
Usage:	business
RAA:	44 %
Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs, trike-trust-level-admin
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	File Attachments, Note Content, Session Tokens, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.	

Outgoing Communication Links: 3

Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target:	Redis Cache
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Session Tokens
Data Received:	Session Tokens

PostgreSQL Connection (outgoing)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target:	PostgreSQL Database
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false

Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content, User Credentials
Data Received:	Note Content, User Credentials

MinIO File Storage (outgoing)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Target:	MinIO Object Storage
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments
Data Received:	File Attachments

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false

Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Route to ECS API (incoming)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Source:	AWS API Gateway
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

AWS API Gateway: 1 / 1 Risk

Description

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Undocumented Entry Point: AWS API Gateway is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@aws-api-gateway](#)

Unchecked

Asset Information

ID:	aws-api-gateway
Type:	process
Usage:	business
RAA:	30 %
Size:	service
Technology:	cloud-api-gateway
Tags:	api-gateway, aws, cloud-native
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	API gateway is the single internet entry point in production. No rate limiting = credential stuffing and cost amplification risk. WAF absence leaves XSS/SQLi traffic unfiltered before it reaches the API.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Route to ECS API (outgoing)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Target:	API Server
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

AWS ECR Container Registry: 2 / 2 Risks

Description

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Internet-Reachable Datastore: AWS ECR Container Registry can be reached from an internet-exposed asset with no application-tier separation: Exploitation likelihood is *Likely* with *Very High* impact.

[internet-reachable-datastore@container-registry](#)

Unchecked

Undocumented Entry Point: AWS ECR Container Registry is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@container-registry](#)

Unchecked

Asset Information

ID:	container-registry
Type:	datastore
Usage:	devops
RAA:	9 %
Size:	service
Technology:	container-registry
Tags:	aws, ecr, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts

Data Stored: Build Artifacts
Formats Accepted: File

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Container registry is the final source of truth for deployed images. Tampering with registry contents (image swap or layer injection) propagates malicious code to every deployment. No signing verification means a registry compromise is undetectable until damage is done.

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Push Image to Registry (incoming)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Source: GitHub Actions Build Pipeline
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Received: Build Artifacts
Data Sent: none

Pull Images from ECR (incoming)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Source: ECS Container Platform

Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	true
Data Received:	none
Data Sent:	Build Artifacts

AWS SES: 1 / 1 Risk

Description

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Undocumented Entry Point: AWS SES is internet-exposed but has no entry_point_type declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@aws-ses](#)

Unchecked

Asset Information

ID:	aws-ses
Type:	process
Usage:	business
RAA:	0 %
Size:	service
Technology:	mail-server
Tags:	aws, cloud-native, email, ses
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	SES handles PII (email addresses). Sending quota exhaustion prevents delivery of account verification emails, blocking user registration. Email spoofing via insufficient SPF/DKIM/DMARC alignment is possible if domain records are misconfigured.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

SES Email Send (incoming)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Source:	Lambda Email Notifier
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Notification Payloads
Data Sent:	none

Browser SPA: 1 / 1 Risk

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Per-Operation Rate Limiting: custom service **Browser SPA** processes auth data with unthrottled outgoing communication links: Exploitation likelihood is *Likely* with *Medium* impact.

[per-operation-rate-limiting-missing@browser-spa](#)

Unchecked

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	29 %
Size:	application
Technology:	browser
Tags:	react, spa, trike-actor, trike-actor-untrusted
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

GitHub Actions Build Pipeline: 1 / 1 Risk

Description

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Undocumented Entry Point: GitHub Actions Build Pipeline is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@build-pipeline](#)

Unchecked

Asset Information

ID:	build-pipeline
Type:	process
Usage:	devops
RAA:	12 %
Size:	service
Technology:	build-pipeline
Tags:	ci, github-actions, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Build pipeline is a privileged component — it has read access to secrets and write access to the container registry. Compromise = full supply chain injection. No SBOM means post-build vulnerability tracking is impossible.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push Image to Registry (outgoing)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Target:	AWS ECR Container Registry
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Build Artifacts
Data Received:	none

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (incoming)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Source:	VaultNote Source Repository
Protocol:	https

Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Received: Build Artifacts
Data Sent: none

LLM Note Summarizer: 2 / 2 Risks

Description

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. **INTENTIONAL:** Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Weak Authentication: internet-exposed entry point **LLM Note Summarizer** lacks strong authentication for confidential data: Exploitation likelihood is *Likely* with *High* impact.

[weak-auth-on-entry-point@llm-summarizer](#)

Unchecked

Low Risk Severity

Missing Threat Scenario: internet entry point **LLM Note Summarizer** has no documented threat scenario: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-threat-scenario@llm-summarizer](#)

Unchecked

Asset Information

ID:	llm-summarizer
Type:	process
Usage:	business
RAA:	20 %
Size:	service
Technology:	llm-inference-endpoint
Tags:	ai, cloud-native, llm, openai
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false

Custom-Developed: true
Client by Human: false
Data Processed: AI Model Responses, Embedding Vectors, Note Content
Data Stored: none
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: LLM endpoint processes user note content. Unauthenticated internet exposure allows arbitrary prompt submission, system prompt extraction via jailbreaks, and inference cost abuse. No guardrails means malicious inputs could generate harmful content or exfiltrate context window data.

Outgoing Communication Links: 2

Target technical asset names are clickable and link to the corresponding chapter.

Query Vector Store (outgoing)

LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Target: VaultNote Vector Store
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Sent: Embedding Vectors
Data Received: Note Content

Call RAG Pipeline (outgoing)

LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Target:	Note RAG Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content
Data Received:	Note Content

Nginx Reverse Proxy: 1 / 1 Risk

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

No Rate Limiting: internet-exposed API **Nginx Reverse Proxy** has communication links without rate limiting: Exploitation likelihood is *Likely with High* impact.

[no-rate-limiting-public-api@nginx-proxy](#)

[In Progress](#)

2026-04-25

Security Team

VAULT-120

Rate limiting via nginx limit_req_zone is planned for sprint 21. Current mitigation: account lockout after 10 failed login attempts implemented in the API layer. Full IP-level rate limiting is a VAULT-120 blocker before production launch.

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	1 %
Size:	service
Technology:	reverse-proxy
Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https

Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

VaultNote Source Repository: 1 / 1 Risk

Description

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Undocumented Entry Point: VaultNote Source Repository is internet-exposed but has no `entry_point_type` declared: Exploitation likelihood is *Likely* with *Medium* impact.

[undocumented-entry-point@source-repo](#)

Unchecked

Asset Information

ID:	source-repo
Type:	process
Usage:	business
RAA:	12 %
Size:	service
Technology:	sourcecode-repository
Tags:	git, github, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Source code is the root of the supply chain. Integrity is critical — malicious code injected here propagates to every built artifact. May contain leaked secrets.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (outgoing)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Target:	GitHub Actions Build Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Build Artifacts
Data Received:	none

VaultNote Vector Store: 1 / 1 Risk

Description

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Internet-Reachable Datastore: VaultNote Vector Store can be reached from an internet-exposed asset with no application-tier separation: Exploitation likelihood is *Likely* with *Very High* impact.

[internet-reachable-datastore@vector-store](#)

Unchecked

Asset Information

ID:	vector-store
Type:	datastore
Usage:	business
RAA:	26 %
Size:	service
Technology:	vector-store
Tags:	ai, criticality-high, embeddings, information-asset-container, qdrant
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Embedding Vectors, Note Content
Data Stored:	Embedding Vectors
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Embedding vectors encode semantic information from user notes. Even without raw text, vectors can be used to infer content via inversion attacks. Unencrypted storage and lack of per-user access control create cross-tenant leakage risk.	

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Retrieve from Vector Store (incoming)

RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Source:	Note RAG Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Embedding Vectors
Data Sent:	Note Content

Query Vector Store (incoming)

LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Source:	LLM Note Summarizer
Protocol:	https
Encrypted:	true
Authentication:	credentials

Authorization: technical-user
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Embedding Vectors
Data Sent: Note Content

AWS RDS PostgreSQL: 0 / 0 Risks

Description

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	rds-postgresql
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service
Technology:	managed-database
Tags:	aws, cloud-native, criticality-high, has-automated-backup, information-asset-container, postgresql, rds
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)

Integrity: critical (rated 4 in scale of 5)
Availability: critical (rated 4 in scale of 5)
CIA-Justification: Production database stores all user PII and note content. Encryption disabled means RDS snapshot exfiltration exposes all data in plaintext. Snapshot sharing misconfiguration (a common AWS incident) bypasses all access controls.

AWS S3 Notes Bucket: 0 / 0 Risks

Description

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	s3-notes-bucket
Type:	datastore
Usage:	business
RAA:	21 %
Size:	service
Technology:	object-storage
Tags:	aws, cloud-native, criticality-high, information-asset-container, s3, third-party-shared
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team
Confidentiality:	confidential (rated 4 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)

Availability: operational (rated 2 in scale of 5)
CIA-Justification: S3 stores user-uploaded files which may contain sensitive documents. No SSE means provider-side access (support channel, storage engineer) reads raw data. third-party-shared because S3 is operated by AWS and subject to subprocessor data protection requirements. No DPA review completed.

ECS Container Platform: 0 / 0 Risks

Description

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	ecs-platform
Type:	process
Usage:	business
RAA:	100 %
Size:	service
Technology:	container-platform
Tags:	aws, cloud-native, ecs, fargate
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	critical (rated 4 in scale of 5)

CIA-Justification: ECS runs the production API containers. Container escape or image tampering grants access to the application runtime and its mounted secrets. No image scanning means known CVEs in base images are undetected until exploitation.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Pull Images from ECR (outgoing)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Target:	AWS ECR Container Registry
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	true
Data Sent:	none
Data Received:	Build Artifacts

Lambda Email Notifier: 0 / 0 Risks

Description

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	lambda-notifier
Type:	process
Usage:	business
RAA:	2 %
Size:	component
Technology:	serverless-function
Tags:	aws, cloud-native, lambda, notifications
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: Lambda processes email addresses and note titles. PII loss due to silent failure creates compliance issues (incomplete processing records). Overpermissive IAM role could be exploited to access other AWS resources.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

SES Email Send (outgoing)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Target:	AWS SES
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Notification Payloads
Data Received:	none

MinIO Object Storage: 0 / 0 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	21 %
Size:	service
Technology:	object-storage
Tags:	criticality-high, information-asset-container, intentional-misconfiguration, minio, object-storage, s3, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: Stores user file uploads which may contain sensitive documents. Default credentials + no encryption = trivial exfiltration by any process on backend-net.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source:	API Server
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments
Data Sent:	File Attachments

Note RAG Pipeline: 0 / 0 Risks

Description

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	rag-pipeline
Type:	process
Usage:	business
RAA:	11 %
Size:	component
Technology:	rag-pipeline
Tags:	ai, langchain, rag
Internet:	false
Machine:	container
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	Embedding Vectors, Note Content
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)

Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	RAG pipeline is the injection point between user-controlled note content and the LLM context window. Without input validation, a malicious note can manipulate the LLM into ignoring instructions, leaking system prompts, or generating harmful content on behalf of an attacker.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Retrieve from Vector Store (outgoing)

RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Target:	VaultNote Vector Store
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Embedding Vectors
Data Received:	Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Call RAG Pipeline (incoming)

LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Source:	LLM Note Summarizer
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false

Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Note Content
Data Sent: Note Content

PostgreSQL Database: 0 / 0 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	postgresql-db
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service
Technology:	database
Tags:	criticality-high, information-asset-container, postgresql, relational, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	critical (rated 4 in scale of 5)

CIA-Justification: Holds all user PII (email, hashed passwords) and all note content. Strictly confidential — unencrypted disk access exposes all data.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source:	API Server
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content, User Credentials
Data Sent:	Note Content, User Credentials

Redis Cache: 0 / 0 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	22 %
Size:	component
Technology:	database
Tags:	cache, criticality-high, information-asset-container, redis, session-store
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Session Tokens
Data Stored:	Session Tokens
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Session token exposure enables account takeover. Non-persistent by	

default but the container volume could persist RDB snapshots.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source:	API Server
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Session Tokens
Data Sent:	Session Tokens

Identified Data Breach Probabilities by Data Asset

In total **12 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 11 as elevated, 0 as medium, and 1 as low.**

These risks are distributed across **8 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

AI Model Responses: 2 / 2 Risks

LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

ID:	ai-model-responses		
Usage:	business		
Quantity:	many		
Tags:	ai		
Origin:	internal		
Owner:	VaultNote Team		
Confidentiality:	confidential	(rated 4 in scale of 5)	
Integrity:	operational	(rated 2 in scale of 5)	
Availability:	operational	(rated 2 in scale of 5)	
CIA-Justification:	LLM responses summarise user note content. If the model was manipulated via prompt injection, responses may leak or alter content from other notes.		
Processed by:	LLM Note Summarizer		
Stored by:	none		
Sent via:	none		
Received via:	none		
Data Breach:	probable		
Data Breach Risks:	This data asset has data breach potential because of 2 remaining risksStr:		
	Probable: weak-auth-on-entry-point@llm-summarizer		
	Improbable: missing-threat-scenario@llm-summarizer		

Build Artifacts: 4 / 4 Risks

Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

ID:	build-artifacts
Usage:	devops
Quantity:	few
Tags:	supply-chain
Origin:	internal
Owner:	VaultNote Team
Confidentiality:	internal (rated 2 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Build artifacts are the final deliverable of the supply chain. Integrity is critical — tampered artifacts compromise every environment they are deployed to.
Processed by:	AWS ECR Container Registry, ECS Container Platform, GitHub Actions Build Pipeline, VaultNote Source Repository
Stored by:	AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository
Sent via:	Push to Pipeline, Push Image to Registry
Received via:	Pull Images from ECR
Data Breach:	probable
Data Breach Risks:	This data asset has data breach potential because of 4 remaining risksStr:

Probable: [internet-reachable-datastore@container-registry](#)

Possible: [undocumented-entry-point@container-registry](#)

Possible: [undocumented-entry-point@build-pipeline](#)

Possible: [undocumented-entry-point@source-repo](#)

Embedding Vectors: 3 / 3 Risks

High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

ID:	embedding-vectors	
Usage:	business	
Quantity:	many	
Tags:	ai, embeddings	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Embedding vectors can be used in model inversion attacks to reconstruct approximate original text. Cross-user embedding access leaks semantic note content.	
Processed by:	LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store	
Stored by:	VaultNote Vector Store	
Sent via:	Retrieve from Vector Store, Query Vector Store	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 3 remaining risksStr:	

Probable: internet-reachable-datastore@vector-store

Probable: weak-auth-on-entry-point@llm-summarizer

Improbable: missing-threat-scenario@llm-summarizer

Note Content: 7 / 7 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	confidential (rated 4 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, LLM Note Summarizer, Nginx Reverse Proxy, Note RAG Pipeline, PostgreSQL Database, VaultNote Vector Store
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline
Received via:	Route to ECS API, Retrieve from Vector Store, Query Vector Store, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline
Data Breach:	probable
Data Breach Risks:	This data asset has data breach potential because of 7 remaining risksStr:

Probable: internet-reachable-datastore@vector-store

Probable: weak-auth-on-entry-point@llm-summarizer

Possible: per-operation-rate-limiting-missing@api-server

Possible: per-operation-rate-limiting-missing@browser-spa

Possible: undocumented-entry-point@aws-api-gateway

Possible: no-rate-limiting-public-api@nginx-proxy

Improbable: missing-threat-scenario@llm-summarizer

File Attachments: 4 / 4 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Files may contain sensitive documents. Integrity is critical because tampered attachments corrupt evidence or business records. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network — intentional misconfiguration for demo purposes.	
Processed by:	API Server, AWS API Gateway, AWS S3 Notes Bucket, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy	
Stored by:	AWS S3 Notes Bucket, MinIO Object Storage	
Sent via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Received via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 4 remaining risksStr:	
	Possible: per-operation-rate-limiting-missing@api-server	
	Possible: per-operation-rate-limiting-missing@browser-spa	
	Possible: undocumented-entry-point@aws-api-gateway	
	Possible: no-rate-limiting-public-api@nginx-proxy	

Notification Payloads: 1 / 1 Risk

Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

ID:	notification-payloads	
Usage:	business	
Quantity:	many	
Tags:	pii	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Email addresses are PII. Note titles may be sensitive. Loss via DLQ absence creates unauditable data processing gaps.	
Processed by:	AWS SES, Lambda Email Notifier	
Stored by:	none	
Sent via:	SES Email Send	
Received via:	none	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 1 remaining risk:	

Possible: [undocumented-entry-point@aws-ses](#)

Session Tokens: 1 / 1 Risk

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 1 remaining risk:	

Possible: [per-operation-rate-limiting-missing@api-server](#)

User Credentials: 4 / 4 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials
Usage:	business
Quantity:	many
Tags:	none
Origin:	customer
Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, Nginx Reverse Proxy, PostgreSQL Database
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server
Received via:	PostgreSQL Connection
Data Breach:	possible
Data Breach Risks:	This data asset has data breach potential because of 4 remaining risksStr: Possible: per-operation-rate-limiting-missing@api-server Possible: per-operation-rate-limiting-missing@browser-spa Possible: undocumented-entry-point@aws-api-gateway Possible: no-rate-limiting-public-api@nginx-proxy

Trust Boundaries

In total **7 trust boundaries** have been modeled during the threat modeling process.

AI Services

Internal network housing the AI/ML components (vector store, RAG pipeline). The LLM inference endpoint is internet-accessible — it is in the Internet trust boundary. AI components have outbound access to the vector store and receive note content from the API server.

ID: ai-services
Type: network-on-prem
Tags: ai
Assets inside: Note RAG Pipeline, VaultNote Vector Store
Boundaries nested: none

AWS Cloud

AWS production environment (VPC). Contains managed services (RDS, S3, ECS, Lambda, API Gateway). Separated from the dev Docker environment by an air gap — production traffic flows through the API Gateway, never through the Docker compose setup.

ID: aws-cloud
Type: network-cloud-provider
Tags: aws, cloud-native
Assets inside: AWS API Gateway, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS RDS PostgreSQL, AWS S3 Notes Bucket
Boundaries nested: none

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID: application-network
Type: network-on-prem
Tags: none
Assets inside: API Server
Boundaries nested: none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID: dmz
Type: network-on-prem
Tags: none
Assets inside: Nginx Reverse Proxy
Boundaries nested: none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID: data-tier
Type: network-on-prem
Tags: none
Assets inside: MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested: none

External CI/CD

GitHub-hosted CI/CD infrastructure and AWS ECR registry. External to the application runtime but directly connected to it via image pull at deploy time. Trust boundary represents the entire supply chain attack surface.

ID: external-cicd
Type: network-cloud-provider
Tags: supply-chain
Assets inside: GitHub Actions Build Pipeline, AWS ECR Container Registry, VaultNote Source Repository
Boundaries nested: none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID: internet

Type: [network-on-prem](#)
Tags: none
Assets inside: **Browser SPA**
Boundaries nested: none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host

All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp:

Threagile Execution Timestamp: 20260604113022

Model Filename: threagile.yaml

Model Hash (SHA256): 15d123b13753140c59d8c6698da83a704aebcacf211732f0ee2ef3b5931b60ef

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.